



**UNIVERSIDAD
DE GRANADA**

**E.T.S. DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN**

**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

Algorítmica, Grupo C

Guión de Prácticas

**Práctica 4: Algoritmos de Vuelta Atrás (Backtracking) y
de Ramificación y Poda (Branch and Bound)**

Curso 2025-2026

Grado en Informática

Objetivo

El objetivo de esta práctica es que el estudiante aprecie la potencia de los métodos vuelta atrás y de ramificación y poda para resolver problemas, pero también comprenda sus limitaciones. Para ello cada equipo de estudiantes deberá diseñar e implementar un algoritmo de vuelta atrás para resolver uno de los problemas (asignado al azar) que se describen a continuación en la sección 1.

Adicionalmente, todos los equipos deberán diseñar e implementar un algoritmo de vuelta atrás y otro de ramificación y poda para el problema descrito en la sección 2.

1. Problemas

1.1. La Exposición del Museo

Un museo está diseñando una nueva exposición circular en su sala principal. Se dispone de n obras de arte únicas que deben colocarse en n pedestales dispuestos en círculo, de forma que el recorrido del visitante sea continuo y vuelva al punto de partida.

Cada obra tiene una relación temática con las demás. Los técnicos del museo han asignado un “índice de fluidez narrativa” a cada par de obras (un valor entero entre 0 y 100) que representa qué tan bien funciona la transición visual y temática si una obra se coloca justo al lado de la otra.

El valor estético total de la exposición se define como la suma de los índices de fluidez entre cada obra y sus dos vecinas (la de su izquierda y la de su derecha). El objetivo es organizar las obras de tal manera que el valor estético total sea el máximo posible.

Diseñad e implementad un algoritmo de vuelta atrás (backtracking) sencillo y otro que emplee cotas para resolver el problema. Probadlo con los datos disponibles en la plataforma de docencia. Realizad un estudio empírico de su eficiencia.

1.2. Antenas de transmisión

Se han identificado n ubicaciones estratégicas geográficas para instalar antenas de transmisión de datos. Sin embargo, algunas de estas ubicaciones están demasiado próximas entre sí o comparten la misma frecuencia, lo que genera interferencias destructivas.

Se dispone de un mapa de interferencias $m[i][j]$ que dice qué pares de ubicaciones son incompatibles: $m[i][j] = 1$ indica que esas ubicaciones son incompatibles, y $m[i][j] = 0$ indica que son compatibles. Si se instala una antena en la ubicación i y otra en la ubicación j y son incompatibles, la señal se anula mutuamente. Por tanto, para cualquier par de ubicaciones incompatibles, solo se puede activar, a lo sumo, una de las dos.

El objetivo es activar el mayor número posible de antenas simultáneamente sin que se produzca ninguna interferencia en la red.

Diseñad e implementad un algoritmo de vuelta atrás (backtracking) sencillo y otro que emplee cotas para resolver el problema. Probadlo con los datos disponibles en la plataforma de docencia. Realizad un estudio empírico de su eficiencia.

1.3. El templo de las lianas colgantes

Eres un famoso explorador que ha encontrado el Santuario de la Niebla. El suelo de la cámara principal se derrumbó hace milenios, dejando solo una cuadrícula de $n \times n$ pilares de piedra emergiendo de un abismo sin fondo.

El objetivo es ir desde el Pilar de Entrada (posición $(0,0)$) hasta el Pilar del Altar (posición (a,b)) para recuperar el ídolo sagrado.

No se puede saltar entre pilares (están demasiado lejos). Hay que usar las lianas antiguas que cuelgan del techo justo encima de cada pilar. Cada pilar en la posición (i,j) tiene una liana con una longitud exacta, determinada por el valor de la matriz $m[i][j]$. Al agarrar la liana y dejarte llevar, la física del péndulo te transportará exactamente esa cantidad de casillas en línea recta. Si estás en (i,j) y la liana mide k , puedes elegir aterrizar en $(i+k,j)$, $(i-k,j)$, $(i,j+k)$ o $(i,j-k)$. Si un balanceo te lleva fuera de los límites de la matriz (contra una pared o al vacío), esa dirección está prohibida. Una vez usada una liana, esta se rompe. No puedes volver a aterrizar en un pilar que ya has visitado en tu ruta actual.

Por ejemplo para la matriz de tamaño 5×5 siguiente,

2	3	1	4	3
4	1	1	1	2
2	4	3	3	2
1	3	1	3	2
4	3	4	1	1

un camino posible desde la posición $(0,0)$ a la posición $(4,4)$, que da 4 saltos, así como el camino de menor número de saltos (3) son, respectivamente:

$(0,0)$ $(0,2)$ $(0,3)$ $(4,3)$ $(4,4)$, o representándolo matricialmente

1	0	2	3	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	4	5

y $(0,0)$ $(2,0)$ $(4,0)$ $(4,4)$, o representándolo matricialmente

1	0	0	0	0
0	0	0	0	0
2	0	0	0	0
0	0	0	0	0
3	0	0	0	4

Diseña e implementa un algoritmo de vuelta atrás (backtracking) que resuelva este tipo de problemas. Modifícalo para que encuentre una solución con el mínimo de saltos posible. Próbalo con los datos disponibles en la plataforma de docencia. Realiza un estudio empírico de su eficiencia.

NOTA importante para los experimentos

Se proporcionan casos concretos de los problemas con los que *debéis* probar vuestros algoritmos y reportar los resultados.

2. Cámaras de vigilancia

El problema de las cámaras de vigilancia, ya expuesto en la práctica anterior sobre algoritmos voraces, consiste en:

Se desea garantizar la seguridad de un complejo mediante la instalación de cámaras de vigilancia. Cada intersección entre pasillos está numerada del 1 al n , y se conoce también el conjunto de pasillos que unen intersecciones mediante pares (i, j) , del pasillo que une la intersección i con la intersección j . Una cámara instalada en una intersección puede vigilar todos los pasillos que llegan a ella. El objetivo es conseguir instalar el menor número posible de cámaras de modo que todos los pasillos estén vigilados por, al menos, una cámara.

2.1. Tareas

En esta práctica se va a abordar este problema mediante algoritmos de vuelta atrás y de ramificación y poda que garanticen encontrar la solución óptima, no una solución aproximada como en el caso de los algoritmos voraces.

- Diseñad e implementad un algoritmo que utilice la técnica de vuelta atrás y otro que use la de ramificación y poda para resolver el problema. En ambos casos se debe definir una cota local optimista del valor que podría obtenerse al explorar el subárbol con raíz en un nodo dado y utilizarla para podar (comparándola con la cota global), y en el caso del algoritmo de ramificación y poda, también para guiar el proceso de búsqueda (usando la estrategia LC).
- Además de devolver el valor de la solución encontrada (y la selección de cámaras correspondiente), se deben de obtener también resultados relativos a complejidad: número de nodos generados, número de nodos podados y el tiempo empleado en resolver el problema; para el caso de ramificación y poda también el tamaño máximo de la cola con prioridad de nodos vivos.
- Utilizad el algoritmo de exploración exhaustiva proporcionado en la práctica anterior para comprobar, con tamaños de problema pequeños, que las soluciones encontradas por los algoritmos de exploración de grafos son realmente óptimas. Además, probad vuestros algoritmos para los problemas que aparecen en la plataforma de docencia, reportando los resultados obtenidos y el tiempo de ejecución.

ENTREGA

Se debe de entregar una memoria *detallada* con todas las tareas realizadas (de las dos partes de la práctica). En todos los casos se debe especificar claramente, además de los algoritmos, la representación de la solución, las restricciones explícitas e implícitas, el árbol de exploración, así como en su caso las funciones de factibilidad y las cotas a utilizar. Asimismo se darán detalles del estudio empírico realizado en cada parte. Se debe entregar el código de todos los programas desarrollados. El código tiene que ser coherente con la forma de representación especificada. El informe debe entregarse en formato pdf. En el informe deben aparecer los nombres de todos

los miembros del equipo que hayan participado en la realización de la práctica, así como el porcentaje de participación de cada miembro del equipo.